

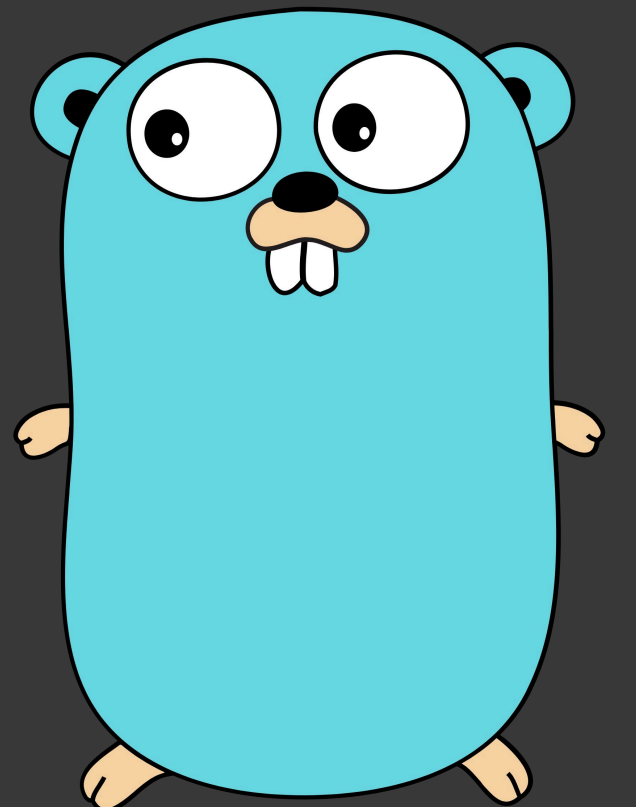
Axelle Lança

Mai 2025



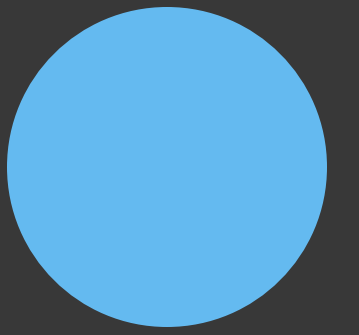
EFREI

GO : Les bases



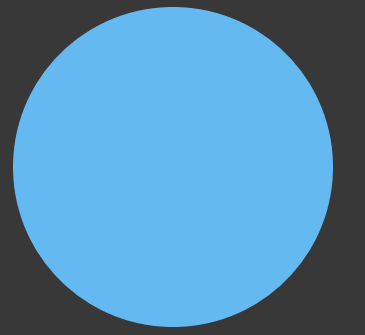
- 
1. Syllabus
 2. Installation
 3. Environnement
 4. Le langage
 5. LA PRATIQUE
 6. Cheat sheets

Qui suis-je



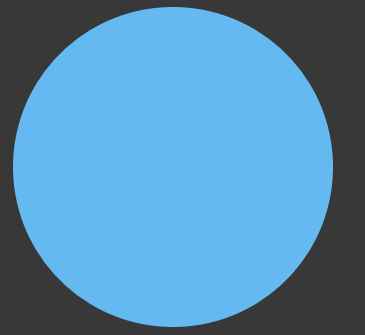
- Axelle Lança
- Développeuse full stack freelance depuis 4 ans
- Pratique du Go depuis 3 ans
- axelle@lanca.fr

Le cours – Objectifs



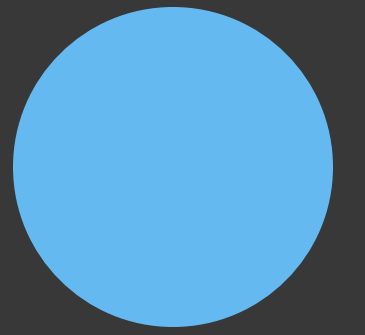
- Comprendre la syntaxe de base de Go
- Écrire, compiler et tester un programme Go
- Savoir structurer un projet Go simple
- Utiliser les outils fournis avec le langage

Le cours – Thèmes abordés



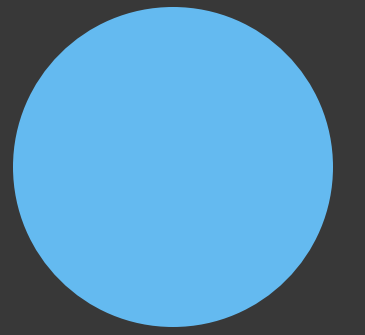
- Introduction et installation
- Syntaxe et types de base
- Fonctions et structures
- Gestion des erreurs
- Tests unitaires
- CLI
- Organisation du code
- Concurrency, channels
- API

Le cours – Autres



- 30 heures
- 4 journées de 7h30
- Notation à définir
 - 4 petits TP (4 × 5 points)
 - Un DM en groupe ou devoir en cours

Installation de Go



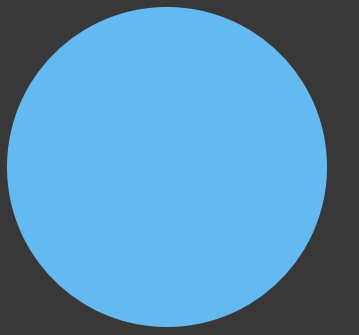
- Pour PC et Linux : <https://go.dev/dl> et suivre les étapes
- Mac : `brew install go`

Vérifier l'installation :

```
go version
// go version go1.24.3 darwin/arm64

go env
// Que voyez-vous ?
// Vérifiez le GOPATH
```

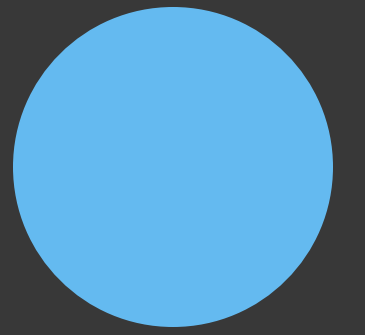
Environnement



`go env`

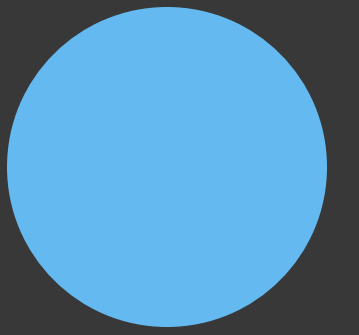
- Go utilise toutes ces variables pour définir son comportement, la gestion des dépendances et la construction des projets
- **GOROOT** : Répertoire racine de l'installation de Go(SDK). Se définit automatiquement
- **GOPATH** : Spécifie l'emplacement de l'espace de travail Go. L'endroit où on stock son code source Go, les packages téléchargés et les exécutables.

Environnement



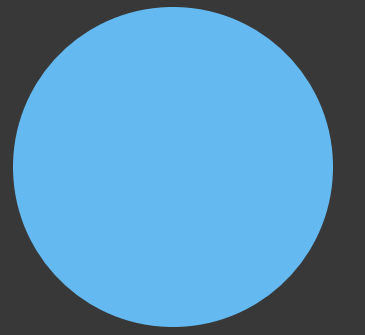
- Linux/macOS : \$HOME/go
- Windows : %USERPROFILE%\go

Environnement



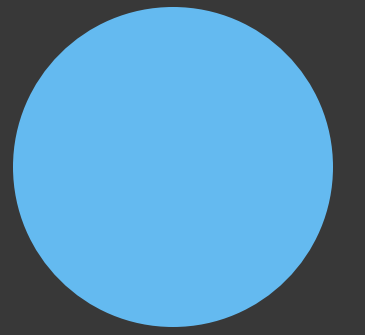
- `$HOME/go/src/FORGE/USER/...`
- `$HOME/go/src/github.com/axellelanca/...`

Installation IDE



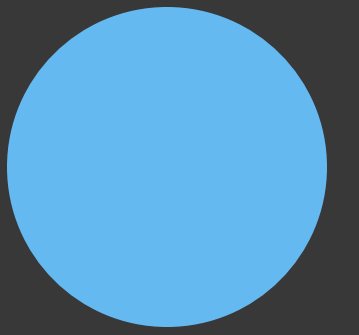
- VSCode
 - Installer l'extension Go via la marketplace
- Goland (IDE JetBrains) (Recommended)
- Playground en ligne : <https://play.golang.org>

Pourquoi go ?



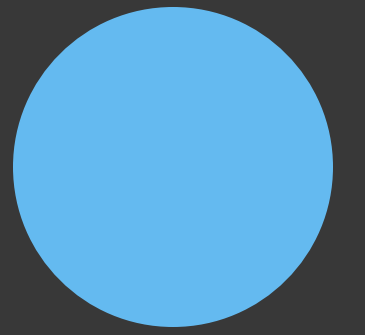
- Conçu chez **Google** en 2007 par Robert Griesemer, Ken Thomson & Rob Pike
- Réponse aux problèmes du **développement à grande échelle** :
 - Compilation lente
 - Gestion complexe des dépendances
 - Code difficile à maintenir
 - Concurrence difficile à gérer
- Particularité principale de Go : Supprimer la contrainte de la gestion de la mémoire et simplifier l'exécution de code en multithreading.

Pourquoi go ?



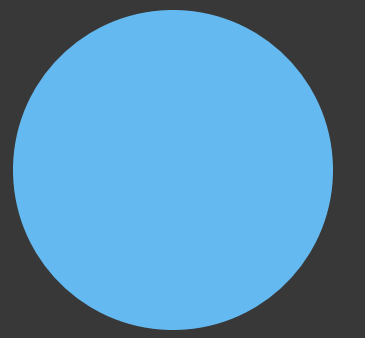
- **Go intègre nativement :**
 - Compilation rapide
 - Formatage du code (gofmt)
 - Tests, benchmarks et fuzzing
 - Documentation automatique
 - Analyse statique du code
 - Gestion des dépendances (go mod)
 - Serveurs de langage
- Simplicité avant tout : 25 mots-clés seulement
- Facile à apprendre, mais problématiques complexes

Un langage parfait pour l'industrie



- Pensé pour les serveurs réseaux et la concurrence
- Optimisé pour les machines multi-coeurs
- Fortement adopté par l'industrie :
 - APIs/RPC
 - CLIs
 - Automatisation & traitement de données
 - Web services, agents, deamons
 - Communauté active et partage de code open-source

Un langage parfait pour l'industrie



I write the following in Go:
(Select all that apply.)

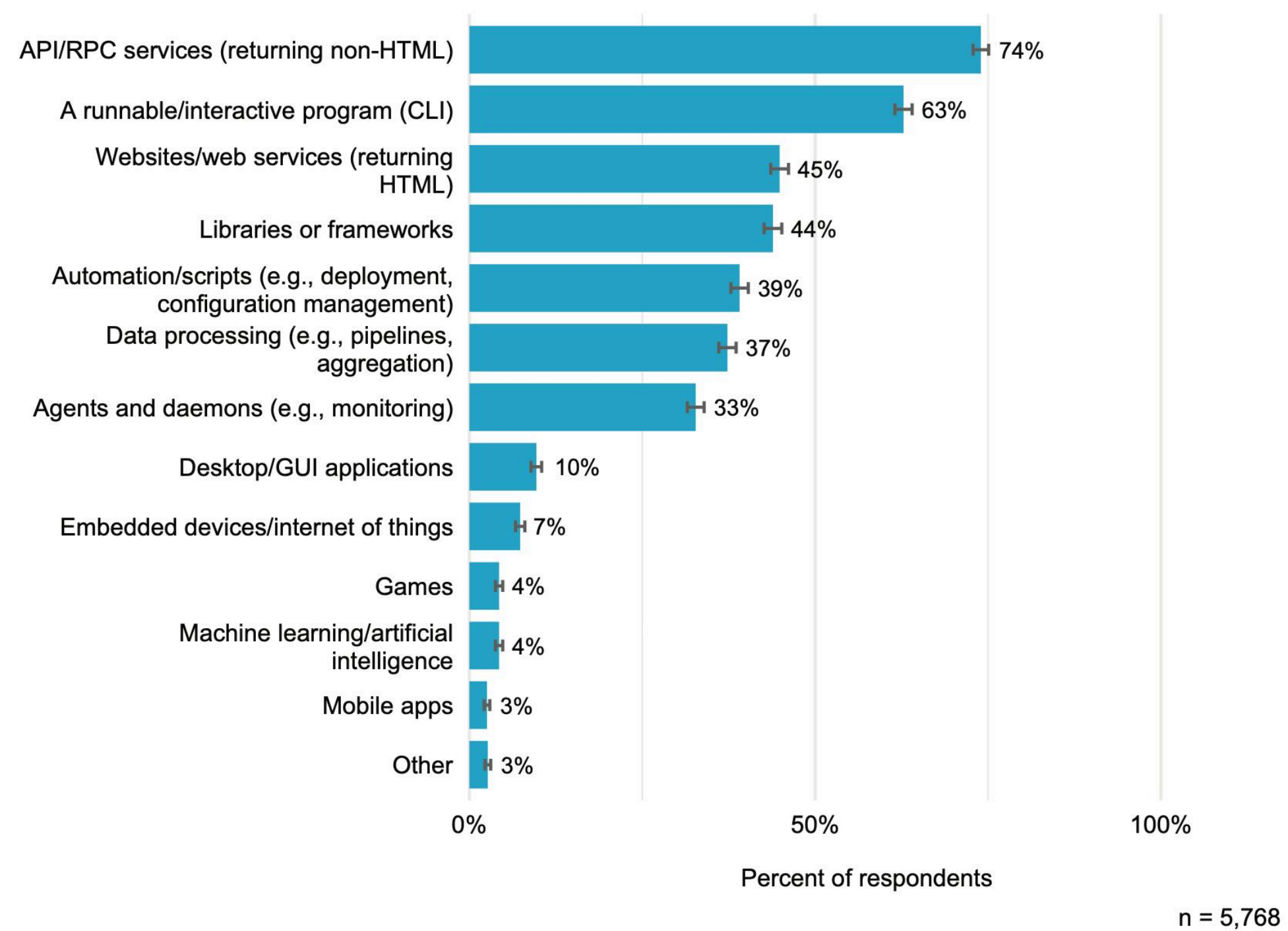
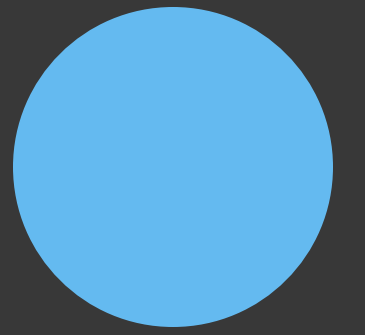


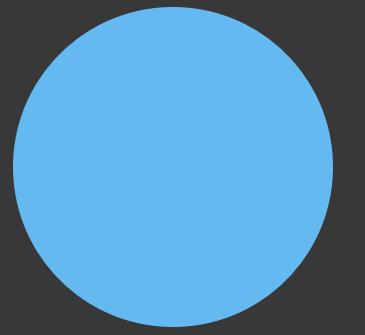
Figure 1.1 Extract from the Go Developer Survey 2024 H1 results

Ce que Go ne fait pas (bien)



- Pas de contrôle total de la mémoire (Garbage collector)
- Les exécutables sont souvent **plus gros que la moyenne**
 - Mais pas gênant en cloud
- Langage jeune, probablement pas aussi développé que Java (par ex.)

Astuces



- Chercher “Go” sur Google = Mission impossible
 - L’astuce : utiliser “Golang” pour les recherches
- Recruter des développeurs Go peut-être difficile... Donc bonne nouvelle !
 - Mais attention pour les juniors : Les problèmes traités avec Go sont complexes

Go et autres langages

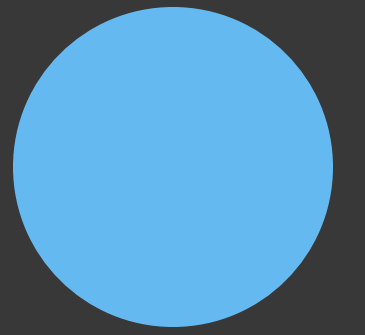
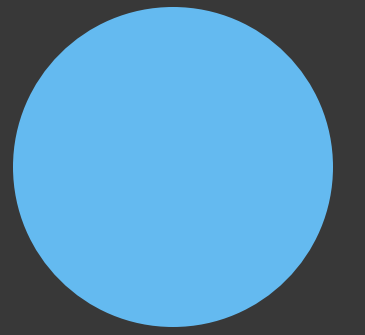


Table 1.1 Comparison of four programming languages

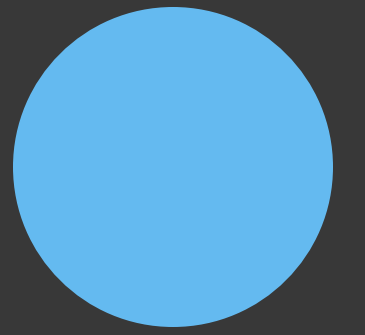
	C++	Python	Java	Go
Design philosophy	High-level object-oriented programming (OOP), procedural, multiparadigm	High-level OOP	High-level OOP	Procedural and data-oriented programming, supports most OOP features
Error management	Via exceptions	Via exceptions	Via exceptions	Errors are values.
Types	Statically typed language	Dynamically typed	Statically typed	Statically typed language
Compilation	Compiled	Compiled at runtime	Compiled Runs in a virtual machine	Compiled
Concurrency	Either memory-level or OS-level threads	OS-level threads	Either memory-level or OS-level threads	Goroutines and channels
Interfaces	Explicit	Implicit and explicit	Explicit	Implicit
Memory release	Full control	Garbage collector	Garbage collector	Garbage collector
Main use cases	High performance, low overhead	Excellent for data analysis	Well-suited for web applications	Adapted for web APIs and cloud computing
Testing tools	External frameworks	Built-in native tools for testing	External framework JUnit	Built-in native tools (test, bench, fuzz)

Structure d'un projet Go



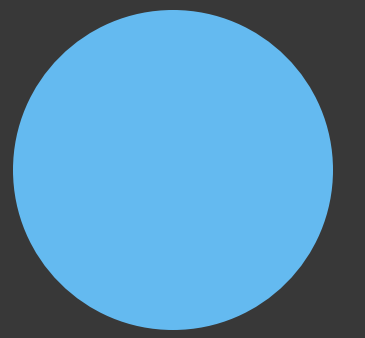
```
monprojet/
├── go.mod           (Définition du module)
├── go.sum           (Sommes de contrôle des dépendances)
├── main.go          (Point d'entrée de l'application [pour un exécutable])
├── internal/        (Code interne non destiné à l'importation externe)
│   └── ...
├── cmd/             (Exécutables spécifiques au projet)
│   └── monoutil/
│       └── main.go
├── api/             (Définitions d'API, souvent pour des services web)
│   └── ...
├── web/             (Ressources web statiques, templates [pour les app web])
│   └── ...
├── scripts/         (Scripts d'automatisation, de build, etc.)
│   └── ...
├── tests/           (Fichiers de test dédiés)
│   └── ...
├── examples/        (Exemples d'utilisation de la librairie)
│   └── ...
├── .gitignore
├── README.md
├── LICENSE
└── ... (Autres fichiers comme des fichiers de configuration)
```

Go en entreprise



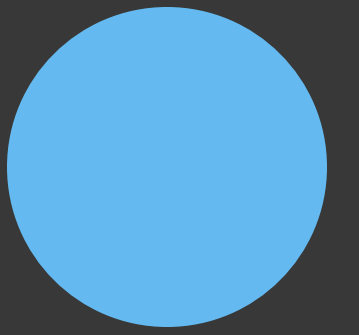
- Uber
- Twitch
- Dropbox
- Docker / Kubernetes
- Dailymotion
- Scaleway
- Heroku
- IbanFirst
- Paypal
- OVH
- LebonCoin
- etc.

En résumé...



Résumé

- Go est un langage moderne, orienté industrie, simple et polyvalent, idéal pour le développement backend, largement utilisé pour les outils orientés cloud, excellent pour les interfaces en ligne de commande (CLI), et même adapté aux systèmes embarqués.
- Sa simplicité et sa philosophie de conception rendent Go facile à apprendre, permettant aux équipes de devenir rapidement productives tout en conservant un code lisible et maintenable.
- Les outils intégrés de Go pour les tests, le benchmarking et le profilage facilitent le développement et garantissent la qualité du logiciel.
- Grâce à son modèle de concurrence léger, basé sur les goroutines et les canaux, Go est un excellent choix pour les applications évolutives et concurrentes.
- La communauté croissante du langage et son adoption dans de nombreuses industries font de l'apprentissage de Go un investissement de carrière pertinent.
- Les projets de petite taille (pocket-sized projects) offrent une approche pratique et progressive pour apprendre les fonctionnalités de Go, de la syntaxe de base aux applications prêtes pour le cloud.



Cheatsheet

Go Formatage Cheatsheet (%fmt)

Spécificateur	Description	Exemple de sortie
%v	Valeur par défaut	true, 123, map[a:b]
%+v	Valeur avec champs nommés	{Name:Axelle Age:30}
%#v	Syntaxe Go	main.User{Name:"Axelle"}
%T	Type de la valeur	int, string
%%	Littéral %	%
%t	Booléen	true
%b	Binaire	1101
%d	Décimal	123
%o	Octal	173
%x / %X	Hexadécimal	7b / 7B
%f	Float (décimal)	3.14
%e / %E	Float scientifique	1.23e+03 / 1.23E+03
%s	String	bonjour
%q	String entre guillemets	"bonjour"
%p	Pointeur	0xc000010230
%c	Caractère	A
%U	Unicode formaté	U+0041



Fonctions fmt



Fonction	Description	Exemple	Sortie
<code>fmt.Print()</code>	Affiche sans saut de ligne	<code>fmt.Print("Hello")</code>	Hello
<code>fmt.Println()</code>	Affiche avec un saut de ligne	<code>fmt.Println("Hello")</code>	Hello\n
<code>fmt.Printf()</code>	Affiche avec un format spécifié (placeholders)	<code>fmt.Printf("Age: %d", 30)</code>	Age: 30
<code>fmt.Sprint()</code>	Retourne une chaîne (ne l'imprime pas)	<code>s := fmt.Sprint("Hi")</code>	s = "Hi"
<code>fmt.Sprintln()</code>	Retourne une chaîne avec saut de ligne	<code>s := fmt.Sprintln("Hi")</code>	s = "Hi\n"
<code>fmt.Sprintf()</code>	Retourne une chaîne formatée (comme <code>Printf</code> mais sans imprimer)	<code>s := fmt.Sprintf("Age: %d", 30)</code>	s = "Age: 30"
<code>fmt.Fprint(w, ...)</code>	Écrit dans un writer (ex : un fichier ou buffer)	<code>fmt.Fprint(os.Stdout, "Hello")</code>	écrit dans stdout
<code>fmt.Fprintf(w, ...)</code>	Format + écrit dans un writer	<code>fmt.Fprintf(os.Stderr, "Error: %v", err)</code>	Error: erreur dans stderr
<code>fmt.Fprintln(w, ...)</code>	Écrit dans un writer avec saut de ligne	<code>fmt.Fprintln(os.Stdout, "Hello")</code>	Hello\n dans stdout